4 Automates à pile

Comme un automate d'états finis, un automate à pile est muni d'un contrôle fini (représenté par un graphe étiqueté dont les nœuds sont les états), et possède une « tête de lecture » qui ne peut qu'avancer (ou rester sur place) sur la bande d'entrée. Il possède de plus une pile (qu'on ne peut accéder que « par le haut ») qui lui sert de mémoire (de taille non bornée, mais à accès restreint).
L'idée de base est qu'un automate à pile effectue une transition en avançant ou non dans la chaîne d'entrée, en changeant d'état et en modifiant le sommet de pile ou une chaîne en haut de pile.

1 Automate à pile strict

1.1 Définition

Définition 1.1 Un **automate à pile** strict (AP) P est défini par : $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ où :
- Q est l'ensemble fini **d'états**,
- Σ est **l'alphabet d'entrée** (fini),
- Γ est **l'alphabet de pile** (fini),
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to \mathcal{P}_{finies}(Q \times \Gamma^*)$ est la **fonction de transition**,
- $q_0 \in Q$ est **l'état initial**,
- $Z_0 \in \Gamma$ est le **symbole initial de pile**,
- $F \subseteq Q$ est l'ensemble des **états finals**.

Étant donné un automate à pile strict, on définit la notion de configuration et la relation de transition permettant ensuite la définition du langage accepté par l'automate.

Définition 1.2 Une **configuration de** P est un élément $(q, w, \alpha) \in Q \times \Sigma^* \times \Gamma^*$ formé d'un état, de la chaîne qui reste à analyser, et de la chaîne en pile.
Une **transition** de P entre les configurations $c = (q, aw, Z\alpha)$ et $c' = (q', w, \gamma\alpha)$, avec $a \in \Sigma \cup \{\varepsilon\}$, $Z \in \Gamma$, $\gamma\alpha \in \Gamma^*$, est possible ssi $(q', \gamma) \in \delta(q, a, Z)$, et on note $c \vdash c'$.
Une configuration initiale (resp. finale) est de forme (q_0, w, Z_0) (resp. (q, ε, α)).
Le **langage accepté** par P avec critère d'état final est :

$$L(P) = \{w \in \Sigma^* | (\exists q, \alpha)[(q_0, w, Z_0) \overset{*}{\vdash} (q, \varepsilon, \alpha) \text{ et } q \in F]\}$$

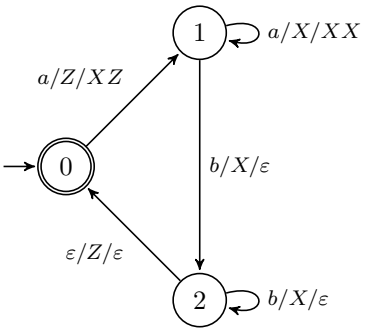
On définit ici directement un automate à pile non déterministe. En effet, on n'a pas, comme pour les automates d'états finis l'équivalence entre automates déterministes et non déterministes. Il se trouve que la classe des langages hors-contexte (HC) est exactement celle acceptée par les automates à pile (non-déterministes).
Remarquons aussi que, dans cette définition, on peut faire une transition sans avancer dans la chaîne d'entrée, par contre, on ne peut pas effectuer de transition lorsque la pile est vide.
Dans la définition qui précède, on a utilisé la convention gauche : on suppose que le «haut» de la pile est écrit à gauche, et c'est pourquoi on écrit :
$(q, aw, Z\alpha) \vdash (q', w, \gamma\alpha)$ si $(q', \gamma) \in \delta(q, a, Z)$.
En convention droite, on aurait écrit :
$(q, aw, \alpha Z) \vdash (q', w, \alpha\gamma)$ si $(q', \gamma) \in \delta(q, a, Z)$.

1.2 Exemples

Exemple E4.18:

Donnons un automate P tel que $L(P) = \{a^n b^n | n \geq 0\}$.

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ où :
- $Q = \{0, 1, 2\}$,
- $\Sigma = \{a, b\}$,
- $\Gamma = \{Z, X\}$,
- l'état initial est 0,
- $Z_0 = Z$,
- $F = \{0\}$.
- $\delta(0, a, Z) = \{(1, XZ)\}$
  $\delta(1, a, X) = \{(1, XX)\}$
  $\delta(1, b, X) = \{(2, \varepsilon)\}$
  $\delta(2, b, X) = \{(2, \varepsilon)\}$
  $\delta(2, \varepsilon, Z) = \{(0, \varepsilon)\}$



L'état 1 correspond à la lecture des a et à l'empilage des X, l'état 2 au dépilage des X et à la lecture des b correspondant. On donne la séquence d'acceptation du mot *aaabbb* sous forme de trace dans la table ci-dessous :

état	**entrée**	**pile**
0	$aaabbb$	Z
1	$aabbb$	XZ
1	$abbb$	XXZ
1	bbb	$XXXZ$
2	bb	XXZ
2	b	XZ
2	ε	Z
0	ε	ε

La démonstration de la propriété annoncée ($L(P) = \{a^n b^n | n \geq 0\}$) est classique : il est facile de démontrer que $\{a^n b^n | n \geq 0\} \subseteq L(P)$ en exhibant une séquence d'acceptation. Pour l'inverse, il faut analyser la structure des séquences menant à une configuration $(0, \varepsilon, \alpha)$.

On remarque que, dans le modèle d'automate à pile strict, on lit un et un seul symbole en sommet de pile, sur chaque transition. Ce modèle manque de souplesse si l'on souhaite dépiler (lire) plusieurs symboles en même temps et/ou si l'on souhaite prendre une transition sans avoir de condition (de lecture) sur la pile. C'est pour cela qu'on introduit un formalisme un peu plus général : les automates à pile étendus. Ce nouveau modèle n'ajoutera rien à la "puissance" du modèle des automates à pile : on ne reconnaîtra pas plus de langages (et pas moins).

2 Automates à pile étendus

On modifie la définition de manière à pouvoir remplacer une chaîne (éventuellement vide) en haut de pile.

2.1 Définition

Définition 2.1 Un **automate à pile étendu** (APE) P est défini par : $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
où les éléments ont la même interprétation que pour un automate à pile stricte, mais δ est plus générale :
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma^* \to \mathcal{P}_{finies}(Q \times \Gamma^*)$

De plus, δ est de domaine fini. On a alors :
$(q, aw, \gamma\alpha) \vdash (q', w, \beta\alpha)$ ssi $(q', \beta) \in \delta(q, a, \gamma)$.

Puisque Γ^* est infini, il faut alors que le domaine de δ soit fini, sinon, on aurait une description infinie, et l'automate perdrait tout son intérêt. D'autre part, il est évident qu'un AP est un APE.

2.2 Équivalence entre AP et APE

Théorème 2.1 Soit P un automate à pile étendu. Il existe un automate à pile strict équivalent P_1, c'est à dire tel que $L(P_1) = L(P)$.

3 Variantes de critère d'acceptation

Indépendamment du modèle d'automate, il existe 2 variantes à la définition du langage accepté (critère d'acceptation). Nous avons déjà vu le **langage accepté avec critère d'état final** (définition 1.2). Nous donnons maintenant le **langage accepté avec critère de pile vide**.

Définition 3.1 Soit P un automate à pile (étendu ou non). Le langage accepté avec critère de pile vide est défini par :

$$L_\varepsilon(P) = \{w \in \Sigma^* \mid \exists q \ (q_0, w, Z_0) \overset{*}{\vdash} (q, \varepsilon, \varepsilon)\}$$

On voit que l'ensemble F des états finals ne joue plus aucun rôle.

3.1 Équivalence (d'expression) entre critères d'acceptation

En général, $L(P) \neq L_\varepsilon(P)$. Cependant, les deux critères sont équivalents au sens précisé par le théorème suivant.

Théorème 3.1 Soit P un AP. Il existe deux automates P' et P" tels que :

1. $L(P) = L_\varepsilon(P')$
2. $L_\varepsilon(P) = L(P'')$

En d'autres termes, pour un même automate, les deux critères n'acceptent pas le même langage, mais ces deux critères ont le même pouvoir d'expression (chacun permet de définir la classe des langage hors contextes HC).

4 Grammaires hors-contexte et automates à pile

On peut construire des automates «à la main» à partir de la caractérisation d'un langage, mais ce qui fait l'intérêt des automates est que l'on peut aussi les construire à partir de grammaires hors-contexte. Ainsi, chaque grammaire hors-contexte admet 2 **automates canoniques**, l'un ascendant, l'autre descendant.
Soit $G = (N, T, P, S)$ une grammaire hors contexte.

Définition 4.1 L'**automate canonique descendant** de G est l'automate $Desc = (Q, \Sigma, \Gamma, \delta, q_0, S, \emptyset)$ où :

- $Q = \{q_0\}$,
- $\Sigma = T$,
- $\Gamma = N \cup T$,
- S (la source de G) est le symbole initiale de pile.
- il n'y a pas d'état final (l'automate sera utilisé uniquement avec critère de pile vide).
- pour chaque règle $A \to \alpha$ de P, $(q_0, \alpha) \in \delta(q_0, \varepsilon, A)$ (prédictions)
- pour chaque $a \in T$, $(q_0, \varepsilon) \in \delta(q_0, a, a)$ (lectures)

Cet automate est défini en utilisant la convention «gauche», i.e. le haut de la pile est à gauche.
On a $L_\varepsilon(Desc) = L(G)$, i.e. le langage décrit par la grammaire est le langage de $Desc$ avec critère de pile vide.

Définition 4.2 L'**automate canonique ascendant** de G est l'automate $Asc = (Q, \Sigma, \Gamma, \delta, q_0, \$, F)$ où :

- $Q = \{q_0, q_1\}$,
- $\Sigma = T$,
- $\Gamma = N \cup T \cup Z_0$ (Z_0 n'appartient pas aux vocabulaires de la grammaire),
- $F = \{q_1\}$.
- le symbole initial de pile (\$) est un symbole qui n'apparaît pas dans la grammaire.
- pour chaque règle $A \to \alpha$ de P, $(q_0, A) \in \delta(q_0, \varepsilon, \alpha)$ (réductions)
- pour chaque $a \in T$, $(q_0, a) \in \delta(q_0, a, \varepsilon)$ (lectures, appelées aussi glissements)
- $(q_1, \varepsilon) \in \delta(q_0, \varepsilon, \$S)$

Cet automate est défini en utilisant la convention «droite», i.e. le haut de la pile est à droite.
On a $L_\varepsilon(Asc) = L_f(Asc) = L(G)$, i.e. le langage décrit par la grammaire est le langage de Asc avec critère de pile vide, ou d'état final (équivalent dans cet automate particulier).

Exemple E4.19: Automate canonique descendant

Soit G la grammaire des expressions arithmétiques sur {a, b}.

E → E+T | T
T → T*F | F
F → (E) | a | b

L'automate canonique descendant de G est : $Desc = (Q, \Sigma, \Gamma, \delta, q_0, S, \emptyset)$ où :

- $Q = \{q_0\}$,
- $\Sigma = \{a, b, +, *, (,)\}$,
- $\Gamma = \{a, b, +, *, (,), E, T, F\}$,
- E est le symbole initial de pile.
- transitions de prédictions :
  $\delta(q_0, \varepsilon, E) = \{(q_0, E + T), (q_0, T)\}$
  $\delta(q_0, \varepsilon, T) = \{(q_0, T * F), (q_0, F)\}$
  $\delta(q_0, \varepsilon, F) = \{(q_0, (E)), (q_0, a), (q_0, b)\}$

- transitions de lecture :
  $\delta(q_0, a, a) = \{(q_0, \varepsilon)\}$
  $\delta(q_0, b, b) = \{(q_0, \varepsilon)\}$
  $\delta(q_0, +, +) = \{(q_0, \varepsilon)\}$
  $\delta(q_0, *, *) = \{(q_0, \varepsilon)\}$
  $\delta(q_0, (, () = \{(q_0, \varepsilon)\}$
  $\delta(q_0,),)) = \{(q_0, \varepsilon)\}$

On donne une trace d'acceptation de la chaîne "a+b*a" dans la table ci-dessous :

état	**entrée**	**pile**	**commentaire**
q_0	a+b*a	E	configuration initiale
q_0	a+b*a	E+T	prédiction 1
q_0	a+b*a	T+T	prédiction 2
q_0	a+b*a	F+T	prédiction 4
q_0	a+b*a	a+T	prédiction 6
q_0	+b*a	+T	lecture
q_0	b*a	T	lecture
q_0	b*a	T*F	prédiction 3
q_0	b*a	F*F	prédiction 4
q_0	b*a	b*F	prédiction 7
q_0	*a	*F	lecture
q_0	a	F	lecture
q_0	a	a	prédiction 6
q_0	ε	ε	lecture (et configuration finale)

On voit le principe de l'automate canonique descendant dans la trace ci-dessus. La pile contient toujours une prédiction de ce qui reste à lire. Si l'on veut construire un arbre syntaxique au fur et à mesure que l'on avance dans l'automate, on voit qu'on construit l'arbre en commençant par sa racine (le E de la configuration initiale).

Exemple E4.20: Automate canonique ascendant

Soit G la grammaire des expressions arithmétiques sur {a, b} identique à l'exemple E4.19.
L'automate canonique ascendant de G est : $Asc = (Q, \Sigma, \Gamma, \delta, q_0, \$, F)$ où :

- $Q = \{q_0, q_1\}$,
- $\Sigma = \{a, b, +, *, (,)\}$,
- $\Gamma = \{a, b, +, *, (,), E, T, F, \$\}$,
- \$est le symbole initial de pile.
- transitions de réduction (reduce) :
  $\delta(q_0, \varepsilon, E + T) = \{(q_0, E)\}$
  $\delta(q_0, \varepsilon, T) = \{(q_0, E)\}$

$\delta(q_0, \varepsilon, T * F) = \{(q_0, T)\}$
$\delta(q_0, \varepsilon, F) = \{(q_0, T)\}$
$\delta(q_0, \varepsilon, a) = \{(q_0, F)\}$
$\delta(q_0, \varepsilon, b) = \{(q_0, F)\}$
$\delta(q_0, \varepsilon, (E)) = \{(q_0, F)\}$

— transitions de glissement (shift) :
$\delta(q_0, a, \varepsilon) = \{(q_0, a)\}$
$\delta(q_0, b, \varepsilon) = \{(q_0, b)\}$
$\delta(q_0, +, \varepsilon) = \{(q_0, +)\}$
$\delta(q_0, *, \varepsilon) = \{(q_0, *)\}$
$\delta(q_0, (, \varepsilon) = \{(q_0, ()\}$
$\delta(q_0,), \varepsilon) = \{(q_0,))\}$

— transitions d'acceptation :
$\delta(q_0, \varepsilon, \$E) = \{(q_1, \varepsilon)\}$

On donne une trace d'acceptation de la chaîne "a+b*a" dans la table ci dessous. (Vous noterez que, comme nous sommes en convention droite, il est judicieux de mettre la pile à gauche et de l'aligner sur la gauche) :

état	**pile**	**entrée**	**commentaire**
q_0	$	a+b*a	configuration initiale
q_0	$a	+b*a	shift
q_0	$F	+b*a	r6
q_0	$T	+b*a	r4
q_0	$E	+b*a	r2
q_0	$E +	b*a	shift
q_0	$E + b	*a	shift
q_0	$E + F	*a	...
q_0	$E + T	*a	...
q_0	$E + T *	a	...
q_0	$E + T * a	ε	...
q_0	$E + T * F	ε	...
q_0	$E + F	ε	...
q_0	$E + T	ε	...
q_0	$E	ε	...
q_1	ε	ε	acceptation (et configuration finale)

On voit le principe de l'automate canonique ascendant dans la trace ci-dessus. La pile contient les éléments (sous-arbres) reconnus dans ce qui a été lu. Si l'on veut construire un arbre syntaxique au fur et à mesure que l'on avance dans l'automate, on voit que la pile contient les sous arbres du mot lu et qu'on construit l'arbre en partant du bas et en remontant.
La dernière transition sert à accepter le mot lu si le symbole source de la grammaire est le seul symbole restant dans la pile.